Nature of Modern Software Systems

Janaka Balasooriya, Ph.D.
Lecturer, Barrett Honors Faculty
Arizona State University

Ira A. Fulton Schools of Engineering
Arizona State University

Objectives

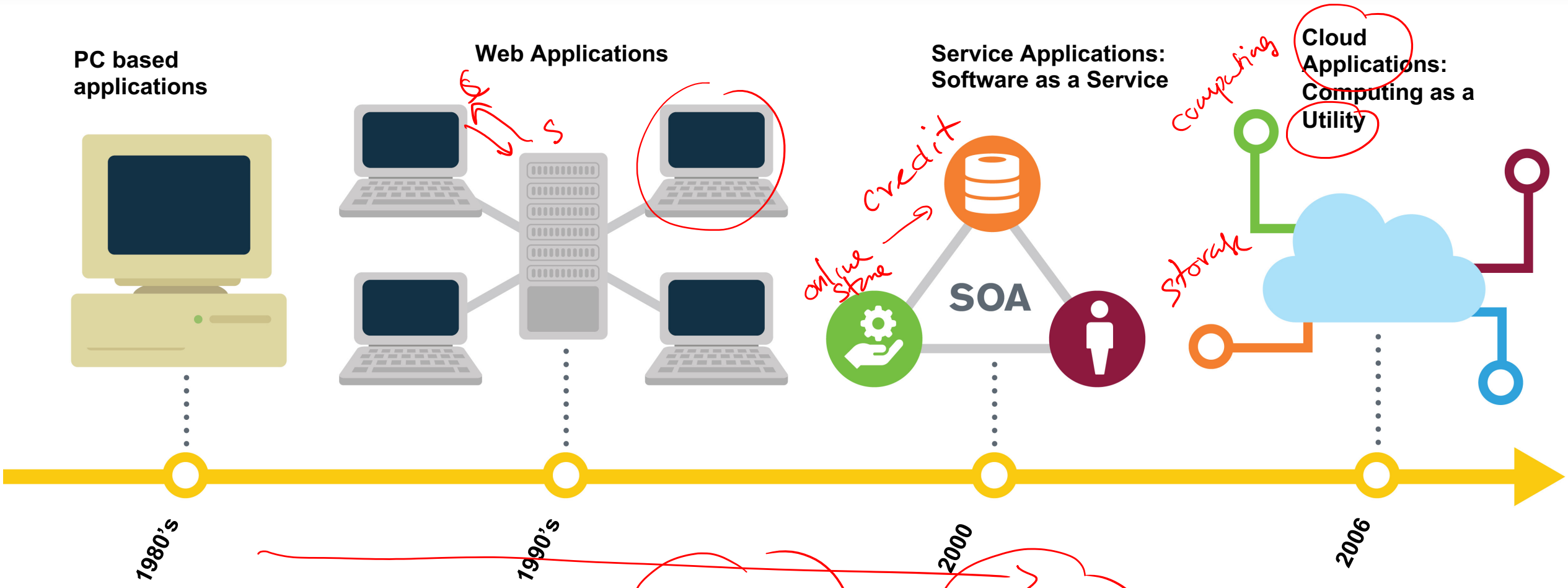
Software in today's world

Explain the role software systems play in today's society.

Software development challenges

Determine high-level challenges in developing today's software systems.

Chronology of Software Systems

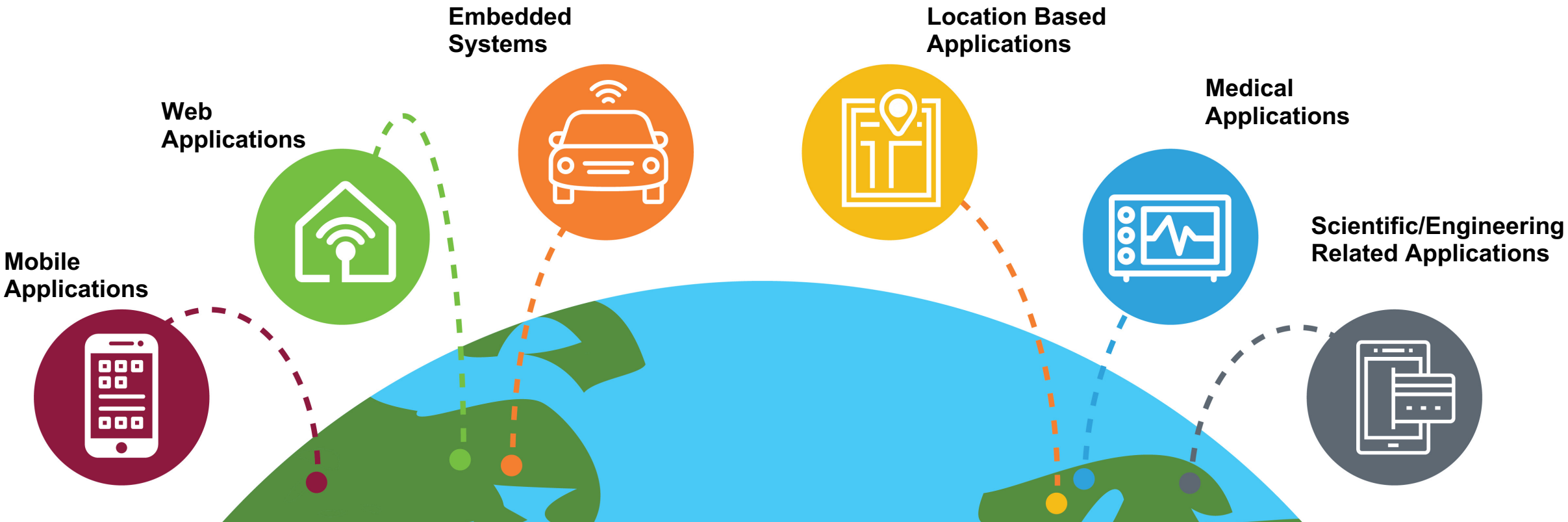


Increases Connectivity and Diversity (heterogeneity)

Software in Today's World

Software systems have become a ubiquitous commodity in today's society.

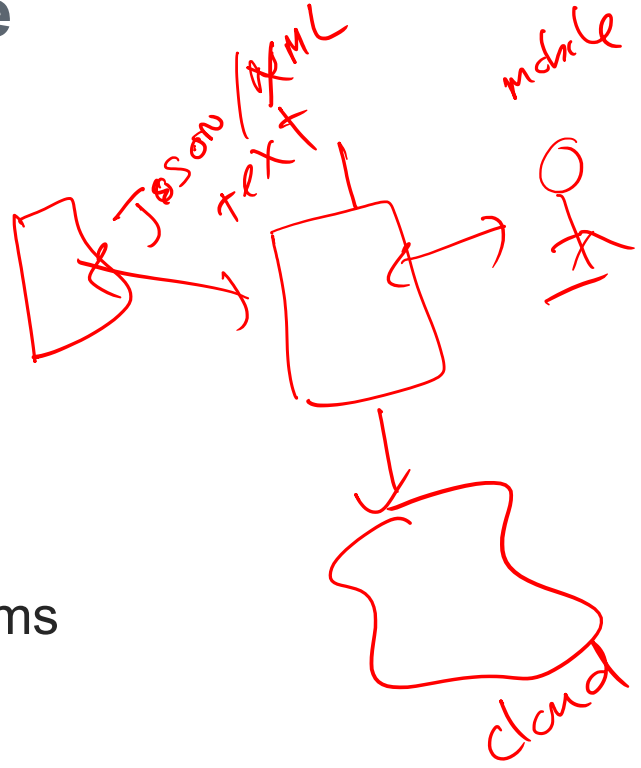
Software systems are everywhere.



Challenges Due to Distributed Nature of Software

Data for software systems may not come from a single place

- Device heterogeneity
  - Sensors (for airplane control)
  - Web generated data
- Platform heterogeneity
  - Different operating systems
  - Different servers
- Data format heterogeneity
  - XML, JSON, plane text, etc.



Connectivity Requirements

- Uses may be a mobile app user, web user, or a desktop user

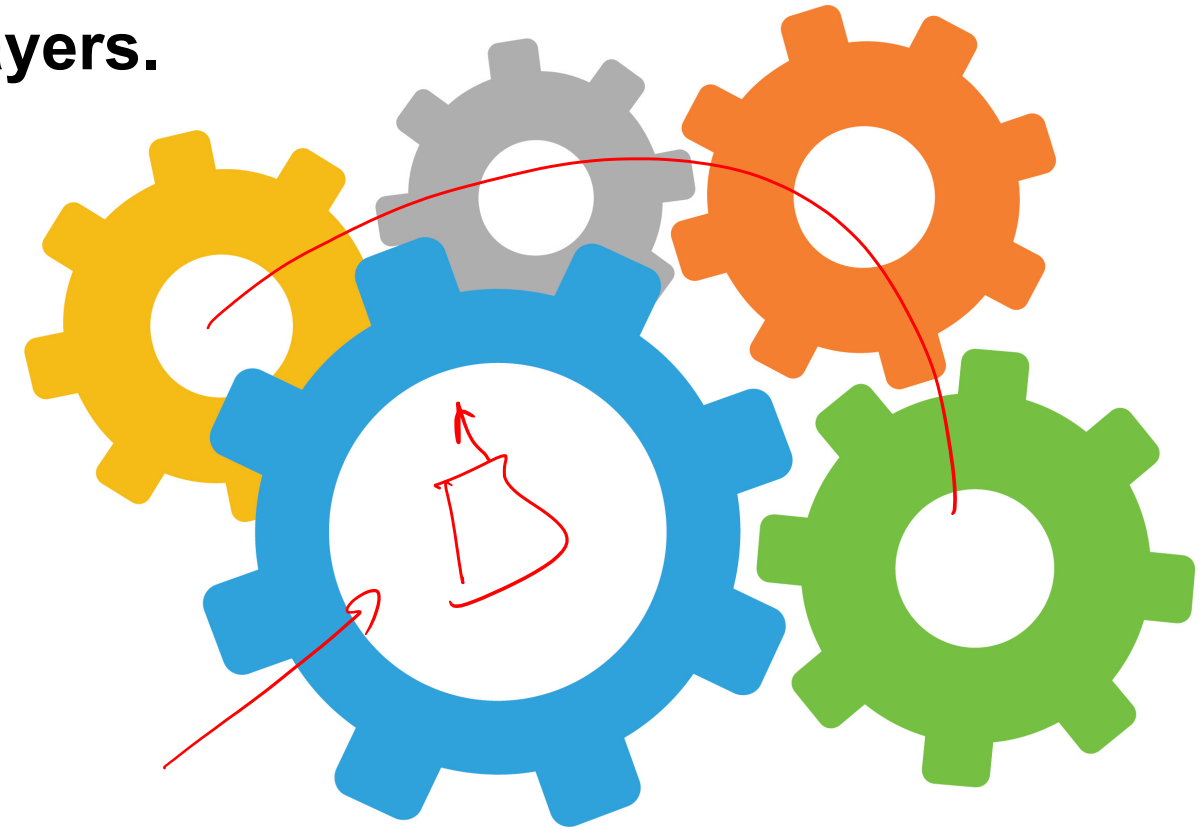
Security, Privacy and Data Integrity Requirements

Resource Requirements

- Process large amount of data, may need cloud resources

Challenges Due to the Size of the Software Systems

Most of today's software systems are built on other software layers.



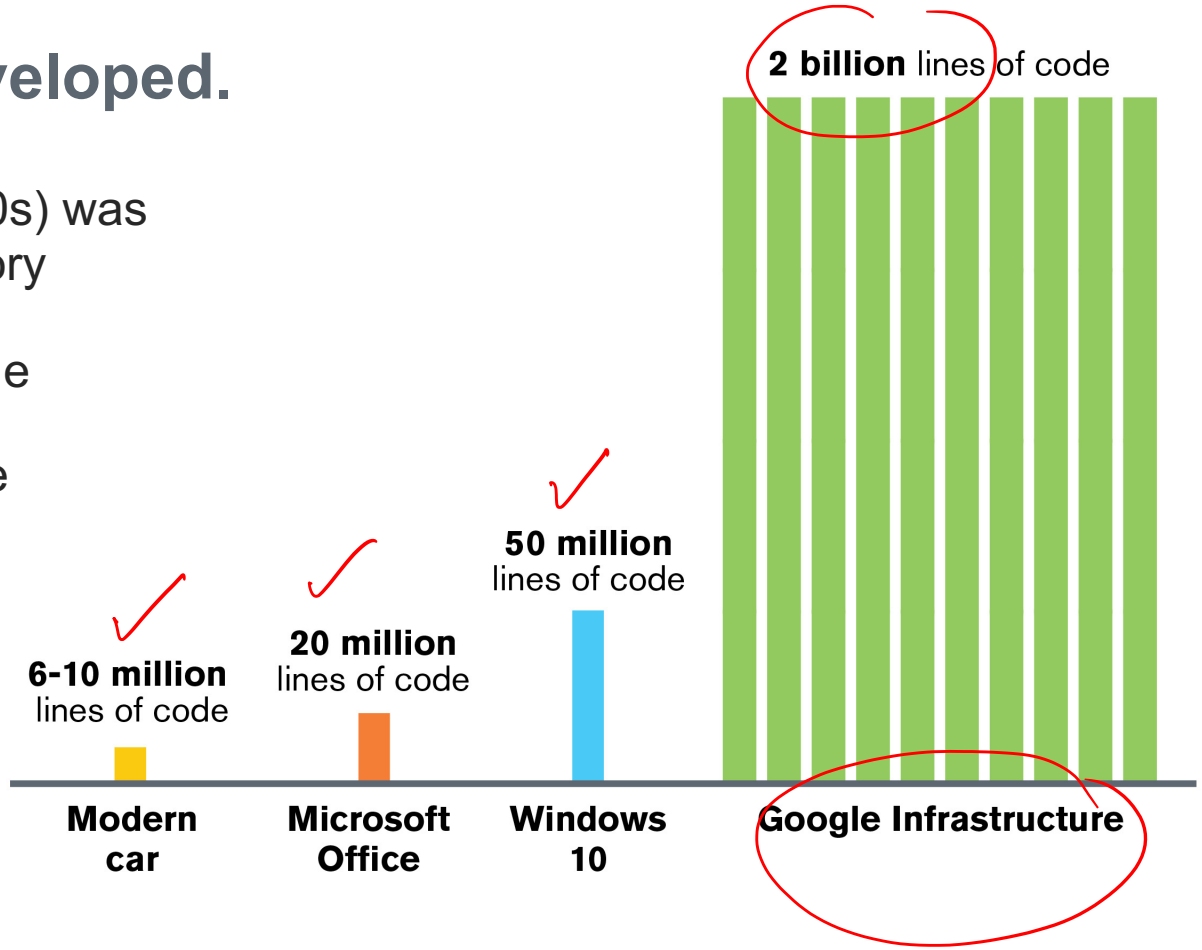
Thus, systems are interconnected.

Challenges Due to the Size of the Software Systems

Requirements are large and computations are complex.

Large amounts of code need to be developed.

- Early versions of DOS operating system (early 80s) was able to reside in a 12 or 28 Kbytes of RAM memory
- A modern car has about **6-10 million** lines of code
- Microsoft Office has about **20 million** line of code
- Windows 10 needs 1 GB of RAM and 16 GB of hard disk minimum and roughly around **50 million** line of code
- Google Infrastructure is about **2 billion** lines of code



Summary

Today's society increasingly depends on computer or software-controlled capabilities.

- Software needs to maintain high quality to reduce the risk of potentially catastrophic failures.
- As software becomes abundant, challenges of developing durable, high quality software with a large amount of code increases.